Bioinformatics session

4th annual workshop on bioinformatics and variant interpretation in InPreD

https://inpred.github.io/26-09_bioinfo_ws/

2. Containerization

A brief history 🎞️

1979	`chroot` system call (changing the root directory of a process and its children to a new location in the filesystem) in Uni V7 considered as beginning of process isolation
2000	FreeBSD Jails allows administrators to partition FreeBSD computer system into several independent, smaller systems (*jails*) – with the ability to assign IP address for each system and configuration
2001	Linux VServer is jail mechanism that can partition resources, similar to FreeBSD Jails

2004	First public beta of Solaris Containers released - combines system resource controls and boundary separation provided by zones
2005	Open Virtuzzo is operating system-level virtualization technology for Linux which uses patched Linux kernel for virtualization, isolation, resource management and checkpointing
2006	Process Containers, launched by Google, was designed for limiting, accounting and isolating resource usage of collection of processes - renamed to *Control Groups (cgroups)* and merged into Linux kernel
2008	LinuX Containers (LXC) was first, most complete implementation of Linux container manager - implemented using cgroups and Linux namespaces

2011	CloudFoundry started Warden can isolate environments on any operating system running as daemon and providing API for container management
2013	Let Me Contain That For You (LMCTFY) kicked off as open-source version of Google’s container stack - providing Linux application containers
2013	Docker emerged
2016	Singularity (later Apptainer) was released

Docker

What is it? 🔎

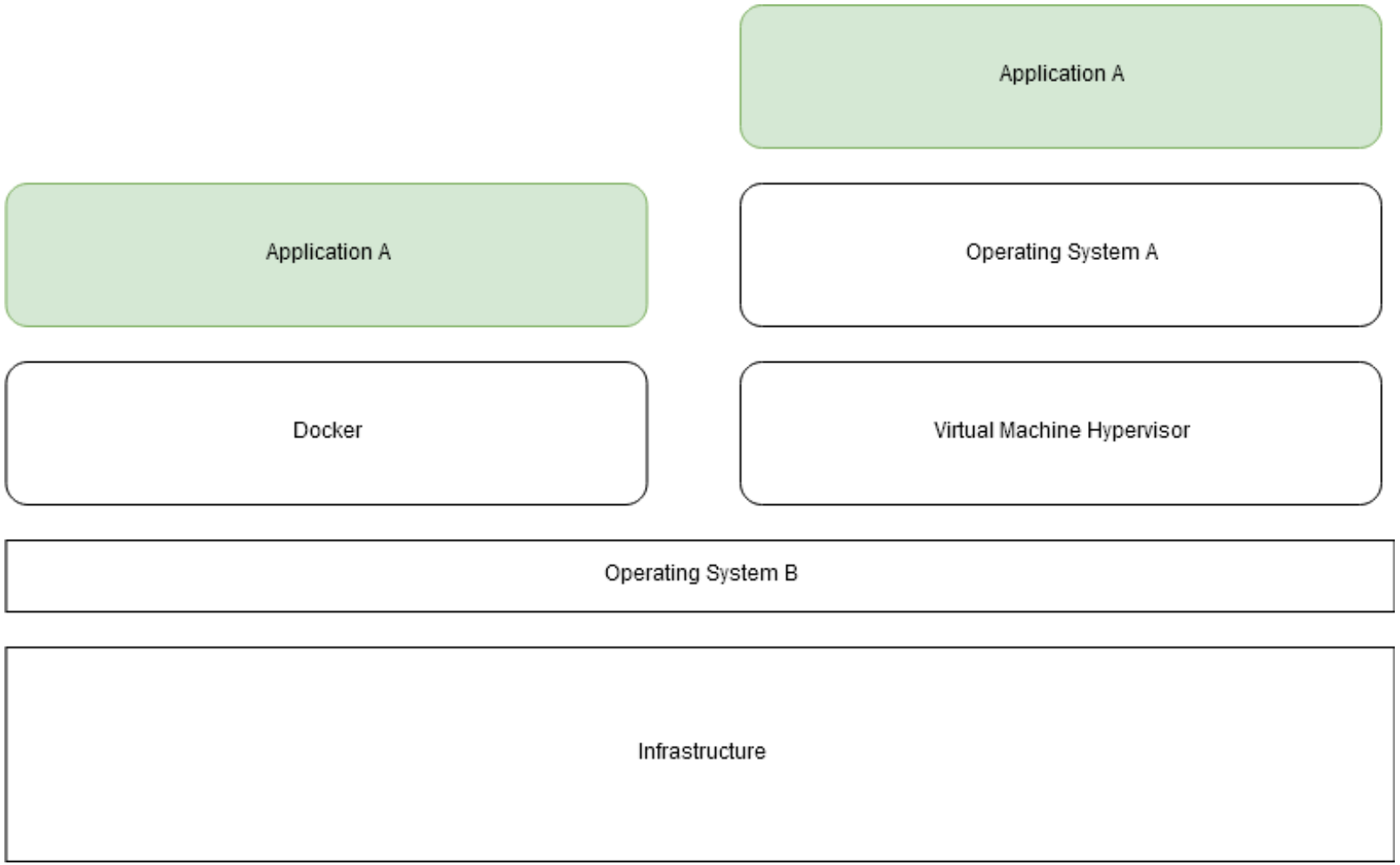
- containerization technology to package application and dependencies into a container
- the container image can be shipped and run consistently across different computing environments

Which benefits does it provide?
🎁

1. Consistency
2. Isolated environments
3. Portability
4. Efficiency

How is it different from virtual machines? 💻

- containers use and share host OS's kernel making them more lightweight and efficient
- virtual machines emulate entire physical machine, including OS making it possible to run multiple OS instances on single physical machine


Application A
Application A
Operating System A
Docker
Virtual Machine Hypervisor
Operating System B
Infrastructure

Key Concepts 🔑

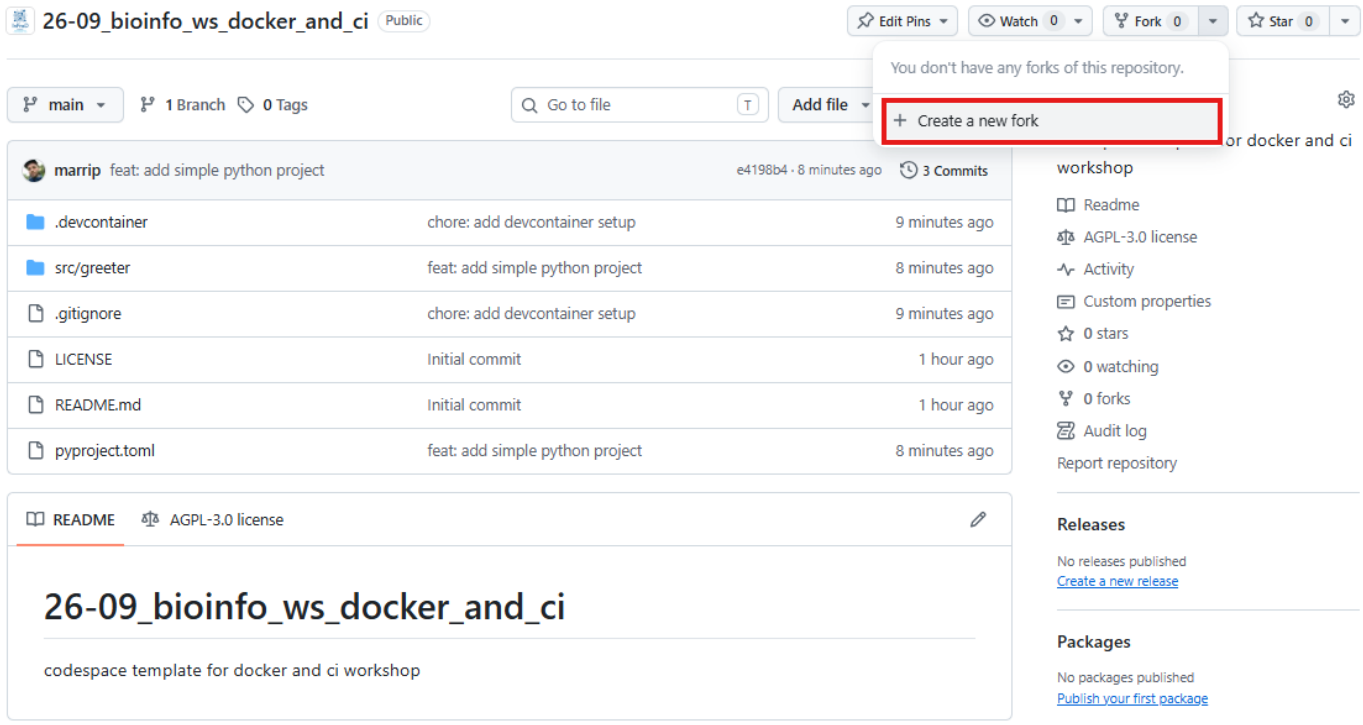
1. **Image:** standardized package that includes all files, binaries, libraries, and configurations to run container
2. **Container:** running instance of image providing isolated runtime environment
3. **Dockerfile:** instructions on how to build image
4. **Docker Hub:** public registry where developers can share and access pre-built images

Key Components 🔑

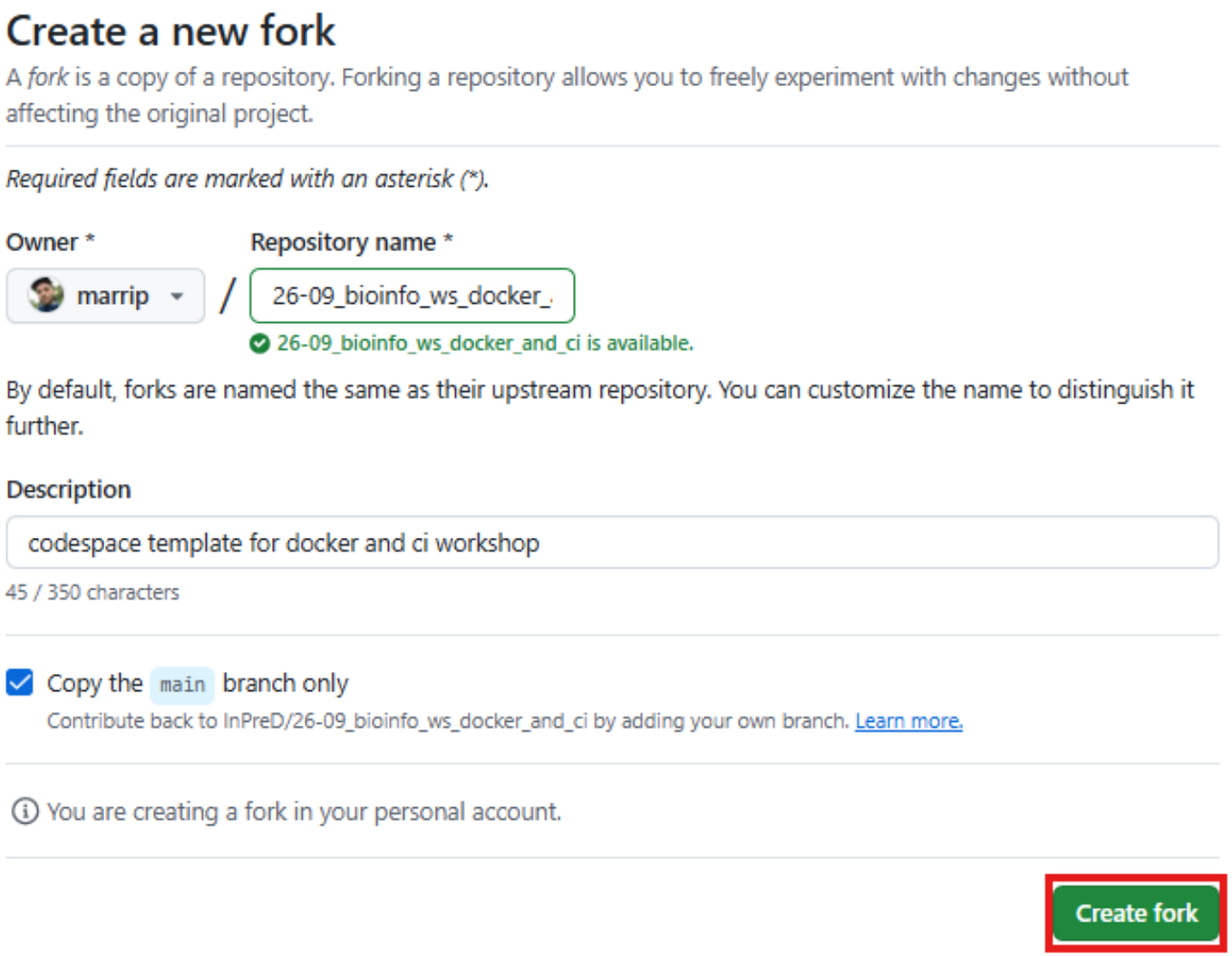
1. **Docker Engine:** core, open-source technology that builds and runs containers
2. **Docker Client:** command-line tool that sends instructions to the Docker Daemon using REST APIs
3. **Docker Daemon (`dockerd`):** receives and processes API requests and calls container runtime
4. **Container Runtime (`containerd`):** turns static container image into running, isolated application on host OS; industry standard

Let's explore! 🗺️

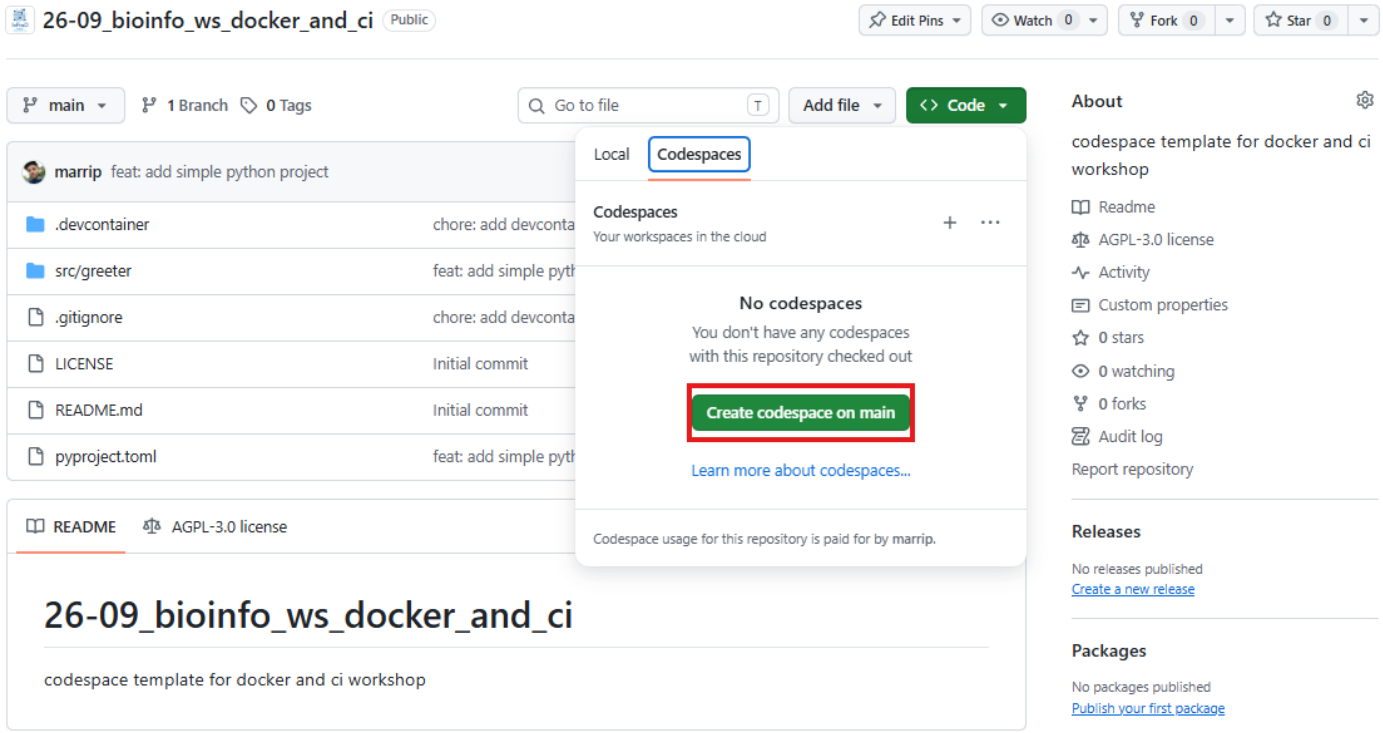
Start by going to https://github.com/InPreD/26-09_bioinfo_ws_docker_and_ci and create a fork



Create your own fork by clicking on `Create fork`



In the forked repository, navigate to `Code` > `Codespaces` > `Create codespace on main`



Run the following commands:

```
# check for running docker containers
$ docker ps
# check for existing images
$ docker images
# pull image
$ docker pull ubuntu:26.04
# run image
$ docker run ubuntu:26.04
```

Run an interactive container:

```
# start interactive container
$ docker run -it ubuntu:26.04
# print container os version
@ cat /etc/lsb-release
# exit container
@ exit
# print os version
$ lsb_release -a
```

Instruct docker to remove containers after use:

```
# check for all docker containers
$ docker ps -a
# remove stopped container
$ docker rm <container hash>
# run docker with --rm flag
$ docker run -it --rm ubuntu:26.04
# exit container
@ exit
# check for all docker containers
$ docker ps -a
```

Remove the docker image

```
# remove docker image
$ docker rmi ubuntu:26.04
# alternatively
$ docker rmi <container image hash>
```

Building an image

We start off by creating a `Dockerfile` in the root directory of our repository. We add the following to our file:

```
FROM python:3.14-slim-trixie
RUN echo 'Hello world' > greetings.txt
```

And then we build and run our docker image

```
# build tagged docker image
$ docker build . -t greeter:test
# run tagged docker images
$ docker run --rm greeter:test cat greetings.txt
```

3. Continuous Integration (CI)